به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین اوّل

مدرس: مهدی صفرنژاد — انتشار: ۲۲ بهمن ۹۷

تفکر الگوریتمی

سؤال ۱. [۱۲ نمره] توابع هر دسته را بر اساس پیچیدگی‌شان مرتب کنید. (فقط ترتیب را از کوچک به بزرگ بنویسید؛ نیاز به نوشتن اثبات نیست.)

آ. [۴ نمره]

$$f_۱(n) = \binom{n}{n-۳} \qquad f_۲(n) = ۱۰۰!$$
$$f_۳(n) = \pi^n \qquad f_۴(n) = n\sqrt[۳]{n}$$

پاسخ: با توجه به این که $f_۲(n)$ ثابت و $f_۳(n)$ نمایی است و همچنین $f_۱(n)$ از مرتبه $n^۳$ می‌باشد و به وضوح از $f_۴(n)$ بیشتر است، بنابراین:

$$f_۲(n) < f_۴(n) < f_۱(n) < f_۳(n)$$

ب. [۴ نمره]

$$f_۱(n) = n\sum_{i=۱}^{n} \frac{۱}{\sqrt{i}+\sqrt{i+۱}} \qquad f_۲(n) = n^۵ \cdot ۵^n$$
$$f_۳(n) = ۲^{\sqrt[n]{n^{n+۱}}} \qquad f_۴(n) = \sqrt{۲^{\sqrt{n}}}$$

پاسخ: با توجه به این که قسمت نمایی $f_۲(n)$ از بقیه بیشتر است، بنابراین از بقیه توابع رشد بیشتری دارد. از طرفی $f_۱(n)$ از مرتبه $n\sqrt{n}$ می‌باشد. بنابراین از بقیه که نمایی‌اند کوچکتر است. همچنین به دلیل این که رشد توان $f_۳(n)$ از $f_۴(n)$ بیشتر است، بنابراین داریم:

$$f_۱(n) < f_۴(n) < f_۳(n) < f_۲(n)$$

پ. [۴ نمره]

$$f_۱(n) = ۲^{\log_۲ n^۲} \qquad f_۲(n) = n\sqrt{n}$$
$$f_۳(n) = n\binom{n}{۲} \qquad f_۴(n) = \sqrt{n}\log n$$

پاسخ: می‌دانیم $۲^{\log_۲ n^۲} = n^۲$. همچنین $f_۳(n)$ از مرتبه $n^۳$ می‌باشد و رشد این تابع از بقیه بیشتر است. بنابراین داریم:

$$f_۴(n) < f_۲(n) < f_۱(n) < f_۳(n)$$

نمره‌دهی: در هر زیرمسئله، ترتیب درست ۴ نمره دارد و به ازای هر نابه‌جایی یک نمره کسر می‌شود.

سؤال ۲. **[۱۲ نمره]** با استفاده از درخت بازگشت، پیچیدگی تابع $T(n)$ را در هر قسمت به دست آورید. نوشتن جواب نهایی کفایت می‌کند و نیاز به توضیح و اثبات نیست.

آ. **[۴ نمره]** $T(n) = T(n/۲) + n^۲$

پاسخ: $\Theta(n^۲)$. اگر درخت بازگشت را رسم کنیم، در طبقهٔ اول هزینهٔ $n^۲$ در طبقهٔ دوم هزینهٔ $n^۲/۴$ و به همین ترتیب در طبقهٔ kاُم هزینهٔ $n^۲/k^۲$ داریم. اگر همهٔ این هزینه‌ها را جمع کنیم، جواب یک ضریب ثابت از $n^۲$ می شود.

ب. **[۴ نمره]** $T(n) = ۳T(n/۴) + ۵n$

پاسخ: $\Theta(n)$. تعداد طبقات در درخت بازگشت برابر $\log_۴ n$ است؛ پس در کل تعداد برگ‌هایمان برابر $= ۳^{\log_۴ n}$ $n^{\log_۴ ۳}$ است که از n کمتر است و جمع هزینه‌هایمان در کل طبقات هم برابر می‌شود با $۵n(۱+۳/۴+۹/۱۶+...)$. پس جواب $\Theta(n)$ می شود.

پ. **[۴ نمره]** $T(n) = ۲T(n-۱) + \Theta(n)$

پاسخ: $\Theta(۲^n)$. تعداد طبقات درخت بازگشت برابر n است پس در کل ۲^n تا برگ داریم. جمع هزینه‌هایمان در کل طبقات هم برابر $n + ۲(n-۱) + ۴(n-۲) + ...$ است که برابر است با $\Theta(۲^n)$. پس در کل جواب هم $\Theta(۲^n)$ است.

توضیح سؤال‌های ۴ تا ۶. در جلسهٔ اول با تعدادی از الگوریتم‌های پیدا کردن قله آشنا شدید. در مسئلهٔ پیدا کردن قلهٔ دو بعدی، جدولی از اعداد داریم و هدف مساله پیدا کردن یک قله در جدول هست. قله خانه‌ای از جدول است که مقدار آن از مقدار ۴ خانهٔ همسایه‌اش بیشتر یا مساوی است. پنج الگوریتم برای حل این مسئله در بستهٔ تمرین و در فایل `algoritthms.py` موجود است. وظیفه شما در تمرین‌های ۳ تا ۶ خواندن این الگوریتم‌ها و پاسخ به سؤالات مطرح شده است.

سؤال ۳. **[۱۶ نمره]** درستی هر کدام از الگوریتم‌های ۱ تا ۵ را مشخص کنید. تنها نوشتن «بله» یا «خیر» کافی است و نیاز به نوشتن توضیح نیست. (درستی الگوریتم ۴ در سؤال ۶ بیان شده و برای همین مورد سؤال نیست.)

آ. **[۴ نمره]** آیا الگوریتم ۱ درست است؟

ب. **[۴ نمره]** آیا الگوریتم ۲ درست است؟

پ. **[۴ نمره]** آیا الگوریتم ۳ درست است؟

ت. **[۴ نمره]** آیا الگوریتم ۵ درست است؟

پاسخ: از بین الگوریتم‌های مطرح شده، تنها الگوریتم ۳ غلط می‌باشد. می‌توانید مثال نقضی که به عنوان پاسخ سؤال «مثال نقض» آمده است را امتحان کنید.

سؤال ۴. **[۲۰ نمره]** در این سوال باید پیچیدگی الگوریتم‌ها را بررسی کنید. برای هر کدام از الگوریتم‌ها، مشخص کردن مرتبهٔ زمانی کفایت می‌کند و نیاز به نوشتن توضیحات نیست. (توجه کنید بررسی پیچیدگی زمانی مستقل از درستی یا غلط بودن آن است.)

آ. **[۴ نمره]** کدام گزینه پیچیدگی زمانی الگوریتم ۱ را در بدترین حالت نشان می‌دهد؟

$\Theta(\log n)$ $\Theta(n \log n)$ $\Theta(n^۲)$
$\Theta(n)$ $\Theta(n \log^۲ n)$ $\Theta(۲^n)$

پاسخ: پیچیدگی زمانی الگوریتم ۱ برابر است با $\Theta(n \log n)$.

ب. **[۴ نمره]** کدام گزینه پیچیدگی زمانی الگوریتم ۲ را در بدترین حالت نشان می‌دهد؟

$\Theta(\log n)$ $\Theta(n \log n)$ $\Theta(n^۲)$
$\Theta(n)$ $\Theta(n \log^۲ n)$ $\Theta(۲^n)$

پاسخ: پیچیدگی زمانی الگوریتم ۲ برابر است با $\Theta(n^۲)$.

پ. **[۴ نمره]** کدام گزینه پیچیدگی زمانی الگوریتم ۳ را در بدترین حالت نشان می‌دهد؟

$\Theta(\log n)$ $\Theta(n \log n)$ $\Theta(n^۲)$
$\Theta(n)$ $\Theta(n \log^۲ n)$ $\Theta(۲^n)$

پاسخ: پیچیدگی زمانی الگوریتم ۳ برابر است با $\Theta(n)$. فرض کنید تعداد سطرها برابر m و تعداد ستون‌ها برابر n باشد. در این صورت با توجه به خط ۸۲ و ۸۴ در کد که نشان دهندهٔ این موضوع است که هزینهٔ انجام این کار برابر است با $\Theta(n+m)$، بنابراین رابطهٔ زیر برقرار است:

$$T(m,n) = \Theta(m+n) + T(m/۲, n/۲)$$

به طور بازگشتی، از خود رابطه اصلی در خودش استفاده می‌کنیم، به این ترتیب داریم:

$$T(m,n) = c(m+n) + c(\frac{m+n}{۲}) + c(\frac{m+n}{۴}) + c(\frac{m+n}{۸}) + ...$$

بنابراین از طرفی می‌دانیم که $T(m,n)$ از $۲c(m+n)$ کم‌تر است و هم‌چنین از $c(m+n)$ بیشتر است. بنابراین داریم:

$$T(m,n) = \Theta(m+n)$$

و به طور خاص داریم $T(n,n) = \Theta(n)$.

ت. **[۴ نمره]** کدام گزینه پیچیدگی زمانی الگوریتم ۴ را در بدترین حالت نشان می‌دهد؟

$\Theta(\log n)$ $\Theta(n \log n)$ $\Theta(n^۲)$
$\Theta(n)$ $\Theta(n \log^۲ n)$ $\Theta(۲^n)$

پاسخ: پیچیدگی زمانی الگوریتم ۴ برابر است با $\Theta(n)$.

ث. **[۴ نمره]** کدام گزینه پیچیدگی زمانی الگوریتم ۵ را در بدترین حالت نشان می‌دهد؟

$\Theta(\log n)$ $\Theta(n \log n)$ $\Theta(n^۲)$
$\Theta(n)$ $\Theta(n \log^۲ n)$ $\Theta(۲^n)$

پاسخ: پیچیدگی زمانی الگوریتم ۵ برابر است با $\Theta(n)$. ابعاد ماتریس اولیه را $n \times n$ می‌گیریم. با این فرض، حلقهٔ اول و دوم هر کدام n بار تکرار می‌شوند. همچنین پیدا کردن عنصر بیشینه نیز با تعداد خانه‌ها متناسب است که از $۶n$ کمتر است. همچنین غیر از روند بازگشتی الگوریتم، بقیهٔ اعمال در زمان ثابت انجام می‌گیرند. پس برای عدد ثابت به اندازهٔ کافی بزرگ c داریم:

$$T(n) \leq cn + T(\frac{n-۳}{۲})$$

که نشان می‌دهد الگوریتم از مرتبه n است (می‌توان از استقرا برای اثبات این موضوع استفاده کرد).

سؤال ۵. [۱۲ نمره] در الگوریتم‌های ارائه شده، تعدادی از آن‌ها غلط هستند. برای هر یک از الگوریتم‌های غلط، یک مثال نقض بیابید. مثال نقض خود را مانند مثالی که در فایل `problem.py` نوشته شده‌است، بنویسید و فایل را با اسم `counterex-stdid.py` ذخیره کنید که `stdid` شمارهٔ دانشجویی شما است (به عنوان مثال `counterex-86100000.py`). سپس این فایل را در کوئرا بارگذاری کنید.

پاسخ: تنها الگوریتم غلط، الگوریتم ۳ می‌باشد. ماتریس زیر، یک مثال نقض برای این الگوریتم است.

```
problem_matrix = [
        [0, 0, 9, 8, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 0, 0, 0],
        [0, 2, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
]
```

سؤال ۶. [۲۶ نمره] در میان الگوریتم‌هایی که در اختیارتان قرار گرفته است، اثباتی برای سریع‌ترین الگوریتم درست ارائه دهید. در ادامه، یک اثبات مناسب برای درستی الگوریتم چهارم آورده شده است. شما می‌توانید از الگوی این اثبات برای اثبات خود استفاده کنید.

اثبات الگوریتم چهارم. اثبات درستی الگوریتم ۴ را در دو گام ارائه می‌دهیم. در گام اول ثابت می‌کنیم که الگوریتم ۴ همواره یک خانه را برمی‌گرداند و سپس در گام دوم اثبات می‌کنیم اگر خانه‌ای برگردانده شود حتما قله است. به‌وضوح این دو گزاره درستی الگوریتم را نتیجه می‌دهند.

۱. **این الگوریتم حتما یک خانه را برمی‌گرداند.** برای اثبات درستی این گزاره فرض کنید جدول موجود در یک مرحله $m \times n$ است. حال در هر مرحله بسته به این که تقسیم را از سطر انجام می‌دهیم یا از ستون، ابعاد مسئله به یکی از چهار حالت $m \times \lfloor \frac{n}{۲} \rfloor$ و یا $m \times (n - \lfloor \frac{n}{۲} \rfloor - ۱)$، $(m - \lfloor \frac{m}{۲} \rfloor - ۱) \times n$، $\lfloor \frac{m}{۲} \rfloor \times n$ تبدیل می‌شود. توجه کنید که مقدار m یا n در هر مرحله کم می‌شود و یا این‌که مقداری توسط الگوریتم به عنوان خروجی برگردانده می‌شود. ادعا می‌کنیم که m و n همواره مثبت باقی می‌مانند. بنابر کاهش m یا n در هر مرحله، اگر این ادعا را ثابت کنیم، نشان داده‌ایم که الگوریتم حتماً به اتمام می‌رسد.

برای اثبات این ادعا از برهان خلف استفاده می‌کنیم و فرض می‌کنیم که $n \leq ۰$ (استدلال مشابه برای m برقرار است). ابتدا توجه کنیم که اگر $n > ۲$ باشد، مقدار n در مرحلهٔ بعد نیز مثبت می‌ماند. پس کافی است این موضوع را تنها برای $n = ۱$ و $n = ۲$ بررسی کنیم. همچنین می‌توانیم فرض می‌کنیم در حالت تقسیم ستونی هستیم زیرا در غیر این صورت مقدار n تغییر نمی‌کند.

- اگر $n = ۱$ باشد، آن‌گاه محاسبهٔ بیشینهٔ ستون مرکزی در واقع محاسبهٔ بیشینهٔ کل جدول است که بنابر تعریف یک قله است و برنامه آن را باز می‌گرداند (خانهٔ best_seen نیز در همین ستون است، پس مقدار در این خانه بیش‌تر مساوی مقدار best_seen نیز هست).
- در حالت $n = ۲$ نیز دو حالت ممکن است. اول اینکه خانهٔ بیشینهٔ ستون وسط، یک قله باشد و الگوریتم آن را برگرداند. حالت دوم این است که الگوریتم را برای مسئلهٔ کوچک‌تر شامل best_seen صدا کنیم (اگر مقداری باز نگردد، best_seen حتماً در ستون دیگر قرار دارد). در این حالت نیز $n = ۱$ می‌شود.

پس در دو هر دو حالت n مثبت می‌ماند، که نشان می‌دهد فرض خلف باطل است.

۲. **اگر خانه‌ای برگردانده شود حتماً یک قله (در مسئلهٔ اصلی) است.** فرض می‌کنیم الگوریتم خانهٔ $(r_۱, c_۱)$ را بازگردانده باشد. توجه کنید این خانه در زیرمسئلهٔ خودش حتما یک قله است. حال از برهان خلف استفاده می‌کنیم و فرض می‌کنیم این خانه در مسئلهٔ اصلی قله نباشد. دنبالهٔ مسائل بازگشتی را در نظر می‌گیریم. کوچکترین مسئله را در این دنباله درنظر بگیرید که خانهٔ $(r_۱, c_۱)$ در آن یک قله نیست. در این مرحله، این خانه همسایه‌ای مانند $(r_۲, c_۲)$ دارد که مقدار آن بزرگ‌تر از مقدار پیدا شده است، یعنی $val(r_۲, c_۲) > val(r_۱, c_۱)$. همچنین خانهٔ $(r_۲, c_۲)$ در زیرمسئلهٔ بعدی وجود ندارد؛ پس این خانه حتما روی سطر یا ستونی بوده است که در آن مرحله از روی آن تقسیم انجام شده است. امّا الگوریتم، مقدار best_seen را با توجه به ستون‌ها و سطرهای تقسیم‌شونده مقداردهی می‌کند. اگر خانهٔ best_seen را (r_{bs}, c_{bs}) بگیریم داریم $val(r_۲, c_۲) \leq val(r_{bs}, c_{bs})$. همچنین از آن‌جایی که الگوریتم مقدار $val(r_۱, c_۱)$ را برگردانده است، می‌دانیم $val(r_{bs}, c_{bs}) \leq val(r_۱, c_۱)$. پس داریم

$$val(r_{bs}, c_{bs}) \leq val(r_۱, c_۱) < val(r_۲, c_۲) \leq val(r_{bs}, c_{bs})$$

که تناقض است. پس فرض خلف باطل است و حکم ثابت می‌شود. □

پاسخ: برای اثبات مسئله از الگوی صورت سؤال استفاده می‌کنیم. برای راحتی ابتدا یک تعریف انجام می‌دهیم. قاب را خانه‌های روی سطرهای اول، وسط و آخر و همچنین ستون‌های اول، وسط و آخر در نظر می‌گیریم. حال دو گزارهٔ بالا را برای این الگوریتم ثابت می‌کنیم.

۱. **این الگوریتم حتماً یک خانه را برمی‌گرداند.** برای اثبات درستی این گزاره فرض می‌کنیم اندازهٔ مسئلهٔ اولیه $n \times n$ است. می‌توان دید در هر مرحله ابعاد مسئله به یکی از دو حالت زیر تبدیل می‌شود: $(۲ - \lfloor \frac{n+۱}{۲} \rfloor) \times (۲ - \lfloor \frac{n+۱}{۲} \rfloor)$ و یا $(۲ - \lceil \frac{n+۱}{۲} \rceil) \times (۲ - \lceil \frac{n+۱}{۲} \rceil)$. توجه کنید که مقدار ابعاد ماتریس در هر مرحله کم می‌شود و یا این‌که مقداری توسط الگوریتم برگردانده می‌شود. ادعا می‌کنیم که n همواره مثبت باقی می‌ماند. بنا بر توضیحات قبلی اگر این ادعا را ثابت کنیم، نشان داده‌ایم که الگوریتم حتماً یک مقدار را برمی‌گرداند.

توجه کنیم که اگر $n > ۴$ باشد، n حتما در مرحلهٔ بعد نیز مثبت باقی خواهد ماند. همچنین اگر $n = ۴$ باشد، یا در قاب یک قله پیدا می‌کنیم یا $n = ۱$ خواهد شد (دقت کنیم در هر مرحله، زیرماتریسی که شامل همسایهٔ بزرگ‌تر است، انتخاب

می‌شود، همچنین این همسایهٔ بزرگ‌تر در قاب نیست). اگر $n \leq ۳$ باشد، آنگاه تمامی خانه‌های جدول در قاب قرار می‌گیرند و بیشینهٔ قاب، بیشینهٔ جدول و قله خواهد بود که بازگردانده می‌شود. پس همواره مقدار n مثبت باقی می‌ماند که نشان می‌دهد حتماً یک خانه توسط این الگوریتم بازگردانده می‌شود.

۲. **اگر خانه‌ای برگردانده شود حتماً یک قله (در مسئلهٔ اصلی) است.** فرض می‌کنیم الگوریتم خانهٔ $(r_۱, c_۱)$ را بازگردانده باشد. توجه کنید این خانه در زیرمسئلهٔ خودش حتماً یک قله است و همچنین این خانه بیشینهٔ قاب زیرمسئلهٔ خود است. حال اگر این خانه بر روی محیط قاب نباشد (چهار ضلع قاب) که قله بودن در این زیرمسئله و مسئلهٔ اولیه معادل است و این خانه در مسئلهٔ اصلی هم قله است. پس فرض می‌کنیم که این خانه در محیط قاب باشد. از برهان خلف استفاده می‌کنیم و فرض می‌کنیم همسایه‌ای مانند $(r_۲, c_۲)$ در مسئلهٔ اصلی باشد که مقدار آن از مقدار $(r_۱, c_۱)$ بزرگ‌تر باشد. توجه کنید این خانه لزوماً در قاب مسئلهٔ پیشین قرار دارد. بیشینهٔ قاب مسئلهٔ پیشین را $g = val(r_۳, c_۳)$ می‌گیریم. همچنین می‌دانیم این خانه همسایه‌ای داشته که مقدارش از آن بیشتر بوده و حتماً در قاب مسئلهٔ فعلی قرار دارد؛ این خانه را $(r_۴, c_۴)$ می‌نامیم. می‌دانیم مقدار $(r_۱, c_۱)$ بیشینه در این قاب بوده است، یعنی $val(r_۱, c_۱) \geq val(r_۴, c_۴)$. پس داریم

$$val(r_۱, c_۱) \geq val(r_۴, c_۴) > val(r_۳, c_۳) = g \geq val(r_۲, c_۲)$$

که نشان می‌دهد فرض خلف‌مان غلط است. پس حکم ثابت می‌شود. □

سؤال ۷. [۱۲ نمره] یکی از وب‌سایت‌های اخبار تکنولوژی پس از معرفی آیفون جدید قصد دارد برای جذب مخاطب اقدام به برگزاری یک آزمون پرتاب[1] مهیج کند. آن‌ها یک ساختمان k طبقه ($۱ \leq k \leq ۱۰^۴$) در اختیار دارند و می‌خواهند بدانند آیفون جدید سقوط از حداکثر کدام طبقه را می‌تواند تحمل کند. از آنجا که کوتاه بودن ویدیوی نهایی و سر نرفتن حوصلهٔ کاربران برای این وب‌سایت مهم است، آن‌ها می‌خواهند آزمون سقوط در پرتاب‌های کمتری خاتمه یابد. همچنین با توجه به افزایش قیمت دلار، این وب‌سایت می‌خواهد از حداکثر دو گوشی آیفون در این آزمون‌ها استفاده کند. فرض کنید در نتیجهٔ هر آزمایش یا گوشی کاملا سالم می‌ماند یا نابود می‌شود. تنها در حالتی که گوشی سالم بماند می‌توان از آن برای آزمایش‌های بعدی نیز استفاده کرد. آن‌ها به سراغ شما آمده‌اند تا به آن‌ها کمک کنید.

در بستهٔ تمرین، یک فایل `droptest.py` در اختیار شما قرار داده شده که دارای یک متد `test` است که باید آن را بهینه کنید. این متد دو ورودی دارد: یک شیء `iphone` و یک عدد k (تعداد طبقات ساختمان). شما می‌توانید از شیء `iphone`، متد `drop(floor)` را اجرا کنید که اگر در نتیجهٔ آزمونِ پرتاب از طبقهٔ `floor`، گوشی سالم بماند ۰ و اگر بشکند ۱ برمی‌گرداند. در حالت‌های دیگر (مثل وقتی که `floor` عدد معتبری نباشد یا دو گوشی قبلاً شکسته شده باشند)، این تابع -۱ برمی‌گرداند. متد `test` در نهایت باید پایین‌ترین طبقه‌ای که پرتاب گوشی از آن منجر به شکستن می‌شود را برگرداند و اگر چنین طبقه‌ای وجود ندارد، $k + ۱$ برگرداند.

متد نمونه موجود در بستهٔ تمرین، دارای یک الگوریتم بدیهی است که طبقه‌ها را از پایین به بالا تست می‌کند و اولین طبقه‌ای که با پرتاب گوشی از آن طبقه، گوشی بشکند را برمی‌گرداند. شما باید این متد را بهینه کنید و تعداد آزمون‌ها را کمتر کنید. قبل از ارسال برنامهٔ خود، آن را با استفاده از دستور `python sampletester.py` تست کنید. این تست برنامهٔ شما را در حالتی اجرا می‌کند که یک ساختمان ۱۰۰ طبقه وجود دارد که از طبقهٔ ۹۹اُم به بالا اگر گوشی را پرت کنیم می‌شکند. در نهایت فایل `droptest.py` جدید را در کوئرا ارسال کنید. نمرهٔ شما بسته به این دارد که برنامهٔ شما از چند آزمون استفاده می‌کند.

[1] drop test

پاسخ: کد زیر نمونه‌ای است که نمرهٔ کامل می‌گیرد. به طور خلاصه فرض کنید k کوچک‌ترین عددی باشد که $\binom{k+1}{2} \geq n$. آیفون اول را از طبقات k، $k + (k-1)$، $k + (k-1) + (k-2)$ و ...پرتاب می‌کنیم. به محض این که آیفون اول شکست، طبقات بین بالاترین طبقه‌ای که نتیجهٔ آزمون سقوط سالم بودن است و طبقه‌ای که نتیجهٔ آزمون سقوط شکستن است را یکی یکی امتحان می‌کنیم. با این روش، در مجموع حداکثر از k آزمون استفاده می‌کنیم. به طور مثال مسئله را برای یک ساختمان ۱۰۰ طبقه می‌توان با حداکثر ۱۴ آزمون حل است. می‌توان اثبات کرد این روش از کمترین تعداد آزمون استفاده می‌کند.

```
class DropTest:
    def test(self, iphone, n):
        k = 1
        while (k*(k+1))/2 < n:
            k += 1
        i = k
        l = 1
        while i <= n:
            res = iphone.drop(i)
            if res == 0:
                l = i+1
                k -= 1
                i += max(1, min(k, n-i))
                if i > n:
                    return n+1
            else:
                break
        while l < i:
            res = iphone.drop(l)
            if res == 1:
                return l
            else:
                l += 1
        return i
```

موفق باشید